

EcoHospital Smart

Ciclo CI/CD com Java Spring Boot

Kalicon Amorim da Cruz Souza — RM 563172

FIAP | Atividade acadêmica | 28/09/2026

Código, testes, containerização e configuração de staging/produção.

Build Java, testes, imagem Docker e publicação GHCR verificados no GitHub. Deploy dos dois containers no PC aguarda reinicialização do Windows para concluir WSL.

Documento gerado a partir de docs/documentacao-tecnica.md. Outros integrantes: preencher se houver.

1. Identificação e objetivo

Integrante: Kalicon Amorim da Cruz Souza — RM 563172. Instituição identificada no projeto original: FIAP.
Outros integrantes: não informados; preencher se houver. Data da preparação: 28/09/2026.

Objetivo: demonstrar build automático, execução de testes existentes, containerização e promoção da mesma versão em staging e produção. CI, testes, build Docker, execução de container e publicação GHCR foram executados no GitHub. O usuário escolheu este PC para os dois ambientes; o deploy local aguarda reinicialização do Windows para concluir WSL.

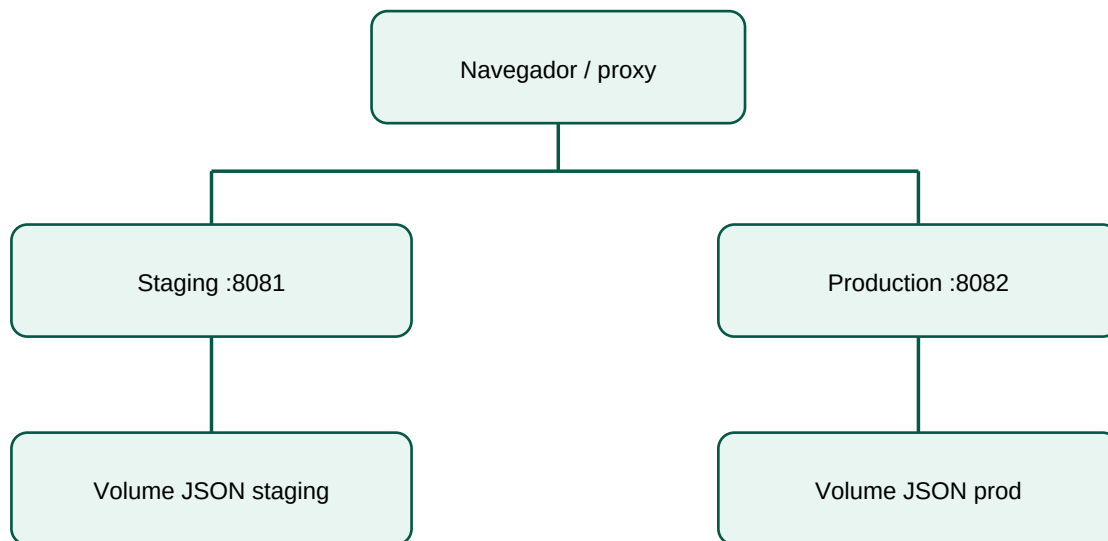
2. Diagnóstico e decisão técnica

O material recebido incluía servidor Node.js nativo, frontend estático, dataset JSON com cinco coleções e scripts acadêmicos MongoDB. Não havia Java/C#, package.json, build, Docker ou CI. A aplicação web simulava consultas MongoDB em arrays e perdia alterações ao reiniciar.

Após autorização para adotar a solução adequada à atividade, foi implementado backend Java 17 com Spring Boot 3.5.16. O frontend e as operações ESG foram mantidos: hospitais, fontes polimórficas, licenças, auditoria, KPIs, consultas, atualizações, simulação IoT e reset. O servidor Node original foi preservado como referência e o runner original continua sendo executado no CI.

Persistência: JSON com gravação atômica em volume por ambiente. Essa escolha mantém o modelo existente com mudanças limitadas e sem serviço de banco artificial. Não se afirma integração MongoDB; seus scripts e prints históricos pertencem à atividade anterior.

3. Arquitetura Docker



Projetos, redes e volumes separados; mesma imagem por digest

Arquitetura prevista: navegador -> proxy/rede do laboratório -> container Spring Boot em cada ambiente -> volume JSON correspondente. O Compose cria uma rede bridge e um volume nomeado por projeto. Staging usa projeto ecohospital-staging e porta 8081; produção usa ecohospital-production e porta 8082. Ambos expõem internamente 8080.

Staging e produção recebem a mesma imagem GHCR por digest SHA256, mas têm configuração e dados separados. Podem estar em dois servidores ou no mesmo host para demonstração, sempre com projetos, portas e volumes distintos. Por padrão, portas no host são limitadas a 127.0.0.1.

O volume contém state.json. O dataset original fica dentro do JAR e é usado somente na criação inicial ou no reset explícito. Reinícios e substituição do container mantêm os dados. Apenas uma instância pode escrever em cada volume. Não há promessa de escalabilidade distribuída, transações MongoDB ou alta disponibilidade.

4. Imagem criada e execução

Dockerfile de dois estágios. Build: maven:3.9-eclipse-temurin-17 executa mvn verify, incluindo JUnit, e gera esg-app.jar. Runtime: eclipse-temurin:17-jre-jammy com JRE, JAR e curl; usuário UID/GID 10001, porta 8080 e healthcheck em /health. Maven e Node não entram no runtime.

O .dockerignore permite apenas pom, Java/testes, frontend e dataset necessário. Compose configura root filesystem somente leitura, tmpfs /tmp, volume de dados gravável, memória de 512 MB, no-new-privileges e capabilities removidas.

Comandos: docker build --build-arg APP_VERSION=academic-local -t ecohospital:local .; docker compose up -d --build --wait --wait-timeout 180. Health esperado: status UP, environment configurado, version da imagem. O CI construiu e executou a imagem; esses comandos locais ainda aguardam o engine Docker deste PC.

5. Etapas e lógica do pipeline

Gatilhos: pull request, push na main ou execução manual. verify executa checagem JavaScript, runner histórico com asserções e Maven verify. O JAR e os XML JUnit são guardados como artefatos.

image depende de verify aprovado. Constrói a imagem, executa container, confere usuário, health, versão e persistência após reinício. Na main, publica a imagem testada no GHCR usando GITHUB_TOKEN; o digest real é passado aos dois deploys. Pull requests não publicam nem fazem deploy.

staging depende da imagem e de DEPLOY_ENABLED=true. production depende de imagem e staging e exige PRODUCTION_DEPLOY_ENABLED=true. O Environment production deve ter aprovação configurada no GitHub. Falha em teste, container ou health de staging bloqueia produção pelas dependências needs e pelo código de saída das etapas.

Modo escolhido: staging-pc e production-pc usam deploy-pc.yml, Docker Desktop Linux e runner Windows com label ecohospital-lab. Variables LOCAL_DEPLOY_ENABLED e LOCAL_PRODUCTION_DEPLOY_ENABLED ficam false até o PC estar pronto. O runner local não executa PRs: exige main e actor Kalicon. Para repositório público, usar runner efêmero somente para jobs confiáveis, nunca código externo. Production tem revisão obrigatória e ambos os Environments estão restritos à main. O pull local usa GITHUB_TOKEN temporário com packages:read, sem PAT permanente.

Cada deploy usa secrets do seu Environment: SSH_HOST, SSH_USER, SSH_PORT, SSH_PRIVATE_KEY, SSH_KNOWN_HOSTS, GHCR_USER e GHCR_TOKEN. Variables: APP_PORT, BIND_ADDRESS e APP_URL. A verificação SSH do host é estrita; senha/token não são argumentos de linha de comando.

O servidor faz pull, executa Compose com --wait e confere ambiente e SHA no health. O runner consulta a URL externa e confere os mesmos campos. Evidência JSON é arquivada somente a partir da resposta real. Produção usa o digest de staging sem recompilar. Deploys do mesmo ambiente não ocorrem simultaneamente.

6. Evidências reais disponíveis

Build Maven: aprovado com JDK 17.0.16. JUnit: 25 casos, zero falhas, zero erros e zero ignorados. Runner original: asserções de consultas/CRUD aprovadas para 50 documentos. Logs e XML em docs/evidence/local.

HTTP local: dois processos Java com a mesma versão SHA256 do JAR responderam 200 na página e health. Staging recebeu uma leitura IoT e ficou com 11 leituras; produção permaneceu com 10. Após reinício, staging manteve 11. Esses resultados são reais, porém locais e sem containers.

Actionlint 1.7.12 aprovou os workflows; Git Bash aprovou sintaxe dos dois scripts de deploy SSH. Isso é análise estática. A execução real posterior do Actions também foi aprovada, incluindo build e smoke test Docker; não houve deploy SSH nem deploy Docker local ainda.

Compose standalone 5.5.1 aprovou config dos ambientes sem daemon. As configurações expandidas confirmam portas 8081/8082, redes e volumes com nomes distintos. Esses recursos ainda não foram criados em Docker. A configuração remota foi validada com imagem/digest de exemplo, sem publicação.

CI atualizado aprovado: <https://github.com/Kalicon/ecohospital-esg-cicd/actions/runs/36433415005> . Commit 5b00e1eff60e82993258fdec04263a57fb462e49. Verify e image aprovados; deploys skipped porque ainda desabilitados. Logs, JSON de jobs e captura real em docs/evidence/github.

Imagem GHCR publicada: ghcr.io/kalicon/ecohospital-esg-cicd@sha256:443ba973654cfd1cf9b0992f6f746d0c24eb1ef9b4d4a4562f2580c4eba03a3d . O job image executou container com UID 10001, health UP e ambiente ci; após uma simulação IoT e reinício, preservou 11 leituras no volume. Esses testes ocorreram no runner Ubuntu, não no PC Windows.

EVIDÊNCIA PENDENTE: página e health do container staging neste PC, porta 8081, ambiente, versão e data. Não foi contratado servidor; localhost não é uma URL pública de hospedagem.

EVIDÊNCIA PENDENTE: aprovação de produção, página e health do container no PC, porta 8082, versão/digest iguais a staging. A regra de revisão existe, mas ainda não ocorreu aprovação nem deploy.

Falha proposital comprovada: <https://github.com/Kalicon/ecohospital-esg-cicd/actions/runs/36433472858> . PR 1 / branch demo/test-gate: uma asserção JUnit falhou; image e todos os deploys ficaram skipped. PR encerrado sem merge, preservando a main com testes aprovados. Captura 02-test-gate-failure.png e logs reais arquivados.

7. Desafios e soluções

Linguagem incompatível com o pedido: backend portado para Spring Boot, mantendo contrato de endpoints e dados. Ausência de asserções no runner histórico: adicionadas verificações que geram código não zero em divergência. Perda de dados no reinício: snapshots e substituição atômica de arquivo por ambiente.

Primeiro teste da página falhou porque MockMvc retorna forward sem renderizar o destino. Corrigido para verificar forward e index.html; também executado HTTP real. A execução final dos 25 testes passou.

Ferramentas indisponíveis: Maven resolvido com Wrapper e checksum; JDK 17 existente selecionado. Actions atualizadas após avisos do primeiro run. Docker Desktop oficial instalado com assinatura Authenticode verificada; WSL 2.7.14 instalado e recursos Windows habilitados após aprovação UAC. Docker registrou WSL_E_WSL_OPTIONAL_COMPONENT_REQUIRED e solicitou reinicialização do PC. Nenhuma reinicialização automática foi feita; deploys não foram declarados concluídos.

Interface melhorada: identificação do ambiente e da versão no painel; requisições HTTP com erro não são mais apresentadas como sucesso; contadores não substituem zero por um valor fictício. Funcionalidades ESG e dataset original permanecem preservados.

8. Operação, limitações e entrega

Sem autenticação nas rotas didáticas de mutação/reset; acesso deve ficar limitado ao laboratório ou proxy protegido. Não usar como serviço público de produção sem autenticação e autorização. Google Fonts é externo, com fallback visual do navegador.

Rollback manual preserva volume e usa previous-image.txt do ambiente. Falha de deploy exige diagnóstico; não há rollback automático, zero downtime ou backup remoto automatizado. Dados devem ser copiados antes de mudanças de formato.

Para concluir: reiniciar Windows manualmente, abrir Docker Desktop e concluir os termos se concordar, verificar engine Linux, preparar runner efêmero, habilitar as variables locais, executar staging, revisar produção e executar production. Inserir capturas reais e atualizar checklist. Não é necessário servidor pago ou SSH para o modo PC; localhost só funciona enquanto este computador e os containers estiverem ligados.

ZIP inclui código Java/Node, dados, frontend, Docker/Compose, workflows, Wrapper, scripts, exemplos e documentação/evidências disponíveis; exclui segredos, ferramentas e estados runtime. O manifesto SHA256 permite conferir conteúdo.

9. Referências oficiais

Spring Boot 3.5 — requisitos: <https://docs.spring.io/spring-boot/3.5/system-requirements.html>

GitHub — environments e proteção:

<https://docs.github.com/en/actions/reference/workflows-and-actions/deployments-and-environments>

GitHub — aprovação e disponibilidade por plano: <https://docs.github.com/en/actions/how-tos/deploy/configure-and-manage-deployments/review-deployments>

Anexo — resultados locais reais

Suíte	Testes	Falhas	Erros	Ignorados
EsgApiTest	20	0	0	0
EsgStoreTest	5	0	0	0

HTTP local — staging

```
{"environment": "staging", "version": "981c75e4d3d7aed96803f575c401992f24867c476780e271385a4bcdac120826", "status": "UP"}
```

HTTP local — production

```
{"status": "UP", "version": "981c75e4d3d7aed96803f575c401992f24867c476780e271385a4bcdac120826", "environment": "production"}
```

Verificado em 2026-09-28T10:55:39.2874294-03:00: HTTP 200 em / e /health dos dois processos; versões SHA256 iguais; dados isolados (11/10); persistência após reinício (11). Demonstração local Java, sem Docker ou deploy remoto.

Os resultados anexados são locais. Não representam evidência de Docker ou deploy remoto.

Anexo — captura real do GitHub

GitHub Actions: verify e image aprovados; deploys ainda desabilitados.

The screenshot shows the GitHub Actions interface for the repository 'Kalicon / ecohospital-esg-cicd'. The workflow 'EcoHospital CI/CD' is displayed, with the title 'Atualiza Actions e prepara deploy Docker em dois ambientes no PC #2'. The status is 'Success', with a total duration of '2m 0s' and '3' artifacts. The workflow diagram shows a sequence of steps: 'verify' (green), 'image' (green), 'staging-pc / deploy' (grey), and 'production-pc / deploy' (grey). The 'verify' and 'image' steps are marked as successful, while the deployment steps are marked as disabled.

Summary

Triggered via push 5 minutes ago

Kalicon pushed [5b00e1e](#) [main](#)

Status	Total duration	Artifacts
Success	2m 0s	3

ci-cd.yml

on: push

```
graph LR; verify[verify] --> image[image]; image --> staging_pc_deploy[staging-pc / deploy]; image --> production_pc_deploy[production-pc / deploy]; staging_pc_deploy --> staging_deploy[staging / deploy]; production_pc_deploy --> production_deploy[production / deploy];
```

Origem: docs/evidence/github/03-updated-ci-success.png. Captura da página real, sem montagem de status.

Anexo — captura real do GitHub

PR descartável: verify falhou; image e deploys bloqueados. PR fechado sem merge.

The screenshot shows the GitHub interface for the repository 'Kalicon / ecohospital-esg-cicd'. The 'Actions' tab is selected, displaying a workflow named 'EcoHospital CI/CD'. The workflow status is 'Failure' with a total duration of '41s' and '1' artifact. The workflow diagram shows a sequence of steps: 'verify' (failed), 'image', 'staging-pr / deploy', 'production-pr / deploy', 'staging / deploy', and 'production / deploy'. The 'verify' step is highlighted with a red 'X' icon, indicating a failure. The workflow is triggered via a pull request, specifically 'demo/test-gate'.

Summary

Triggered via pull request 2 minutes ago

Kalicon opened #1 [demo/test-gate](#)

Status	Total duration	Artifacts
Failure	41s	1

ci-cd.yml

on: pull_request

```
graph LR; verify[verify] --> image[image]; image --> staging-pr-deploy[staging-pr / deploy]; image --> staging-deploy[staging / deploy]; staging-pr-deploy --> production-pr-deploy[production-pr / deploy]; staging-deploy --> production-deploy[production / deploy];
```

Origem: docs/evidence/github/02-test-gate-failure.png. Captura da página real, sem montagem de status.